

그래프(Graph)

그래프(Graph) 개념정리

1. 그래프(Graph)란 무엇인가?

그래프는 **비선형 자료구조**입니다. 배열이나 리스트처럼 순서대로 한 줄로 늘어선 게 아니라, 여러 개의 점들이 서로 자유롭게 연결되어 있는 구조예요.

- **정점(Vertex, V)** — 점 하나하나. 사람, 장소, 섬처럼 "대상" 하나를 의미해요.
- **간선(Edge, E)** — 점과 점을 잇는 선. "관계"나 "이동 가능한 길"을 의미해요.

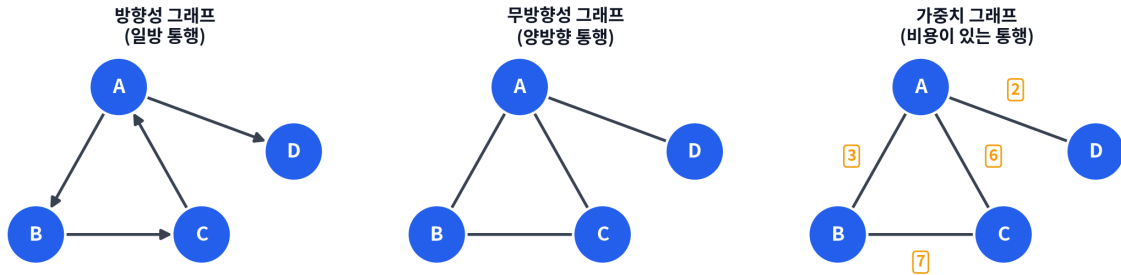
 **비유** — 카카오톡 친구 관계를 떠올려보세요. 사람 한 명 한 명이 정점이고, "서로 친구다"라는 관계가 간선입니다. 지하철 노선도도 마찬가지예요. 역이 정점, 역과 역을 잇는 선로가 간선이죠.

강의자료에 나온 예시처럼, "섬과 다리의 정보가 주어졌을 때 어떻게 저장할 것인가?"라는 질문이 바로 그래프를 공부하는 이유입니다. 섬 = 정점, 다리 = 간선인 셈이죠.

1-1. 그래프의 종류

종류	설명	비유
방향성 그래프	한쪽 방향으로만 이동 가능 (일방통행)	일방통행 도로
무방향성 그래프	양쪽 방향 모두 이동 가능	양방향 도로
가중치 그래프	이동할 때 비용(거리, 시간 등)이 발생	도로마다 통행료가 다른 경우
인접(Adjacent)	A와 B가 간선으로 직접 연결되어 있으면 "인접하다"고 함	바로 옆집 이웃

가중치 그래프는 방향성/무방향성과 별개의 축이에요. "방향이 있으면서 가중치도 있는" 그래프도 얼마든지 가능합니다.



2. 그래프를 컴퓨터에 저장하는 4가지 방법

그래프는 눈에는 점과 선으로 보이지만, 컴퓨터 안에서는 결국 **배열이나 리스트 같은 숫자 뭉치**로 저장해야 합니다. 이때 항상 스스로에게 던져야 할 두 가지 질문이 있어요.

- ? A와 B가 연결되어 있는가?
- ? A와 연결된 정점들은 모두 무엇인가?

이 두 질문에 얼마나 빠르게 답할 수 있느냐에 따라 저장 방식을 골라야 합니다. 방법은 총 4가지이고, 각각 장단점이 뚜렷해서 **문제 상황에 따라 골라 써야** 해요.

2-1. 간선의 리스트 (Edge List)

간선 하나하나를 그냥 리스트에 나열하는 가장 단순한 방법입니다. **문제에서 입력값으로 자주 주어지는 형태**라 반드시 익숙해져야 해요.

```
# [시작정점, 끝정점] 쌍의 리스트
edge = [[0, 1], [0, 2], [0, 5], [0, 6], [4, 3], [5, 3], [5, 4], [6, 4]]
```

- 장점: 입력을 그대로 저장하니 구현이 제일 쉬움
- 단점: "A와 연결된 정점이 뭐지?"를 알려면 리스트 전체를 뒤져야 해서 느림

2-2. 인접 행렬 (Adjacency Matrix)

정점 개수만큼의 정사각형 표(2차원 배열)를 만들고, **행[i][j] = 1** 이면 "i에서 j로 갈 수 있다"는 뜻으로 표시합니다.

```
# mat[i][j] == 1 이면 i -> j 로 가는 간선이 있다는 뜻
mat = [
    [0, 1, 1, 0, 0, 1, 1],
    [0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 0, 0, 0, 0],
]
```

```
[0,0,0,0,0,0,0],
[0,0,0,1,0,0,0],
[0,0,0,1,1,0,0],
[0,0,0,0,1,0,0],
]
```

- 장점: "A와 B가 연결되어 있는가?"를 `mat[A][B]` 한 번 확인으로 **즉시** 알 수 있음
- 단점: 정점이 N개면 항상 $N \times N$ 크기의 표가 필요해서, 정점 수가 많은데 간선이 적은 그래프에서는 메모리 낭비가 심함

2-3. 인접 리스트 (Adjacency List)

각 정점마다 "내가 직접 연결된 정점들"만 리스트로 들고 있는 방법입니다. **그래프 문제에서 가장 많이 쓰이는 방식**이고, 다음 편에서 배울 BFS·DFS도 대부분 이 형태를 기준으로 설명해요.

```
# graph[i] = i번 정점과 연결된 정점들의 리스트
graph = [
    [1, 2, 5, 6], # 0번 정점은 1,2,5,6과 연결
    [0],          # 1번 정점은 0과 연결
    [0],
    [4, 5],
    [3, 5, 6],
    [0, 3, 4],
    [0, 4],
]
```

- 장점: "A와 연결된 정점들이 뭐지?"를 물으면 `graph[A]` 만 보면 바로 끝. 메모리도 실제 간선 수만큼만 사용
- 단점: "A와 B가 연결되어 있는가?"를 확인하려면 `graph[A]` 리스트를 처음부터 훑어야 함 (인접 행렬보다는 느림)

2-4. 그리드 맵 (Grid Map)

미로나 지도처럼 **2차원 격자** 형태의 문제에서 쓰입니다. 사실 이것도 인접 행렬의 일종인데, 칸 하나하나가 정점이고 "0은 길, 1은 벽"처럼 표시해서 상하좌우로 이동 가능 여부를 판단하는 방식이에요.

```
# 0: 지나갈 수 있는 길, 1: 벽
grid = [
    [0, 1, 1, 0, 1, 0, 0, 1],
    [0, 0, 1, 1, 0, 0, 1, 1],
    # ...
]
```

- 이 경우엔 질문이 조금 달라집니다: "A(x,y)와 연결된 칸은 어디인가?", "A(x1,y1)에서 B(x2,y2)로 갈 수 있는가?"

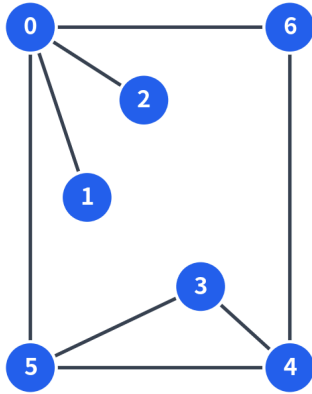
2-5. 네 가지 방법 비교

저장 방식	"A-B 연결 여부" 확인	"A와 연결된 정점 찾기"	메모리 사용량	주로 쓰이는 곳
간선의 리스트	느림	느림	적음	입력값 그대로 받을 때
인접 행렬	빠름	느림	많음 (N^2)	정점 수가 적을 때, 빠른 연결 확인이 중요할 때
인접 리스트	느림	빠름	적음	대부분의 그래프 문제
그리드 맵	빠름(상하좌우만)	빠름(상하좌우만)	칸 수만큼	미로, 지도 문제

💡 중요한 건, 이 네 가지 저장 방식은 **서로 자유롭게 변환**할 수 있어야 한다는 점이에요. 어떤 문제는 간선의 리스트로 입력을 주지만, 풀이는 인접 리스트로 바뀌어서 진행하는 게 훨씬 편한 경우가 많습니다.

같은 그래프를 인접 리스트로 옮기면 실제로 이렇게 됩니다 — 앞서 나온 코드 예시(`graph = [[1,2,5,6], [0], [0], [4,5], [3,5,6], [0,3,4], [0,4]]`)가 바로 아래 그림 그대로예요.

그래프 예시 (무방향)



인접 리스트로 표현하면

정점	인접한 정점
0	1, 2, 5, 6
1	0
2	0
3	4, 5
4	3, 5, 6
5	0, 3, 4
6	0, 4